



Sistema de clasificación automática de subgéneros de música electrónica

Autor:

Ing. Facundo Rivas

Director:

Esp. Ing. Agustín de la Vega (FIUBA)

*Esta planificación fue realizada en el curso de Gestión de proyectos
entre el 21 de octubre de 2025 y el 9 de diciembre de 2025.*

Índice

1. Descripción técnica-conceptual del proyecto a realizar	5
1.1 Contexto del proyecto	5
1.2 Estado del arte	5
1.3 Propuesta de valor	5
1.4 Descripción funcional	5
2. Identificación y análisis de los interesados	7
3. Propósito del proyecto	7
4. Alcance del proyecto	7
5. Supuestos del proyecto.	8
6. Requerimientos	9
7. Historias de usuarios (<i>Product backlog</i>).	11
7.1 Criterio de ponderación	11
7.2 Historias de usuario	12
8. Entregables principales del proyecto	13
9. Desglose del trabajo en tareas	14
10. Diagrama de Activity On Node.	16
11. Diagrama de Gantt	17
12. Presupuesto detallado del proyecto	20
13. Gestión de riesgos	20
14. Gestión de la calidad	23
15. Procesos de cierre	25

Registros de cambios

Revisión	Detalles de los cambios realizados	Fecha
0	Creación del documento	21 de octubre de 2025
1	Se completa hasta el punto 5 inclusive	3 de noviembre de 2025
2	Se completa hasta el punto 9 inclusive	10 de noviembre de 2025
3	Se completa hasta el punto 12 inclusive	18 de noviembre de 2025
4	Se completa el plan	25 de noviembre de 2025

Acta de constitución del proyecto

Buenos Aires, 21 de octubre de 2025

Por medio de la presente se acuerda con el Ing. Facundo Rivas que su Trabajo Final de la Carrera de Especialización en Inteligencia Artificial se titulará “Sistema de clasificación automática de subgéneros de música electrónica” y consistirá en la implementación de un prototipo de un sistema de clasificación de música electrónica. El trabajo tendrá un presupuesto preliminar estimado de 600 horas y un costo estimado de 17815 dólares estadounidenses, con fecha de inicio el 21 de octubre de 2025 y fecha de presentación pública en el mes de agosto de 2026.

Se adjunta a esta acta la planificación inicial.

Dr. Ing. Ariel Lutenberg
Director posgrado FIUBA

Esp. Ing. Agustín de la Vega
Director del Trabajo Final

1. Descripción técnica-conceptual del proyecto a realizar

1.1. Contexto del proyecto

El presente proyecto tiene como objetivo desarrollar un sistema de clasificación automática de subgéneros de música electrónica mediante técnicas de *deep learning*. Actualmente, DJs, productores y profesionales del sector musical dedican considerables recursos de tiempo en la clasificación manual de grandes volúmenes de *tracks* para organizar sus bibliotecas musicales. Este proceso es tedioso, subjetivo y propenso a inconsistencias, lo que representa un desafío significativo para quienes gestionan colecciones extensas de música electrónica.

El proyecto se enmarca como una investigación independiente con aplicación directa en la industria musical, específicamente en el ámbito de la música electrónica de baile. Según datos de Beatport (plataforma líder mundial de distribución y venta de música electrónica, utilizada como referencia por DJs y productores profesionales) de 2023, los géneros *techno* y *house* representan aproximadamente el 60 % del mercado global.

1.2. Estado del arte

En el ámbito de la inteligencia artificial aplicada a la clasificación musical, existen herramientas que permiten distinguir entre géneros musicales generales (rock, jazz, hip hop) donde las diferencias son evidentes. Sin embargo, la distinción entre subgéneros de música electrónica presenta un desafío significativo debido a similitudes en parámetros fundamentales como BPM (*beats* por minuto), estructura rítmica y elementos sonoros.

Trabajos recientes como el de Hsu et al. [1] han demostrado la viabilidad de utilizar arquitecturas *Short-chunk* CNN con ResNet para clasificación fina de subgéneros electrónicos. No obstante, no existe actualmente una solución integrada que combine tanto la clasificación automatizada de alta precisión como una interfaz funcional para uso práctico por parte de profesionales de la música.

1.3. Propuesta de valor

El valor diferencial del proyecto radica en su capacidad de adaptarse específicamente al espectro *techno/house*, combinando un modelo de *Deep Learning* de alta precisión (objetivo: 80 % *accuracy* general, 90 % top-2 *accuracy*) con una aplicación de escritorio funcional. Esto permite automatizar significativamente el proceso de organización de bibliotecas musicales, reduciendo tiempo y recursos, y permitiendo a los profesionales concentrarse en su actividad creativa principal. La arquitectura modular facilita la futura expansión a otros subgéneros mediante *transfer learning*.

1.4. Descripción funcional

La solución propuesta consta de dos módulos principales que trabajan de forma integrada. La figura 1 se presenta el diagrama en bloques del sistema. Se observa que el flujo comienza con la selección de *tracks*, continúa con el análisis mediante inteligencia artificial y finaliza con la organización automática de archivos por subgénero.

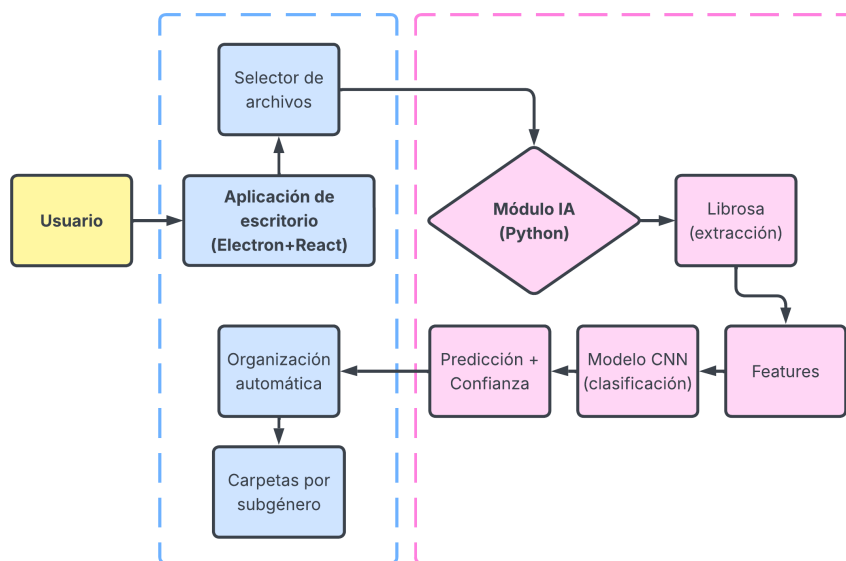


Figura 1. Diagrama en bloques del sistema de clasificación automática.

El sistema clasificará *tracks* en ocho subgéneros específicos: *deep house*, *tech house*, *melodic techno*, *progressive*, *techno peak time*, *hard techno*, *minimal/deep tech* y *trance*. La selección de estos géneros responde a tres criterios: coherencia técnica (rango BPM similar 120-150, estructura *four-on-the-floor*), relevancia industrial (mercado dominante) y disponibilidad de datos validados profesionalmente en plataformas como Beatport y Soundcloud. A continuación, se detallan los módulos del sistema y como están compuestos.

Módulo de Inteligencia Artificial: implementado en Python, utilizará un dataset de 1600 *tracks* (200 por subgénero) en formatos MP3, AIFF y WAV. Mediante la biblioteca Librosa se extraerán características como *mel-spectrograms*, *MFCs*, *tempograms*, *spectral centroid* y *chroma features*. La arquitectura del modelo, basada en *Short-chunk CNN* con ResNet, procesará segmentos de 2 segundos mediante 7 bloques convolucionales con capas residuales, *batch normalization* y técnicas de regularización (*dropout*, *data augmentation*). El entrenamiento utilizará división 70/15/15 para *train/validation/test* con *hyperparameter tuning* y *early stopping*.

Aplicación de Escritorio: desarrollada con Electron y React, proporcionará una interfaz multi-plataforma que permitirá al usuario seleccionar directorios, visualizar el progreso de clasificación en tiempo real y obtener resultados con nivel de confianza. El sistema creará automáticamente subdirectorios por subgénero y moverá los archivos a las carpetas correspondientes.

Las métricas de éxito incluyen: *accuracy* general mayor a 80 %, *top-2 accuracy* mayor a 90 %, *F1-score* balanceado, tiempo de procesamiento menor a 5 segundos por *track* y matriz de confusión para análisis detallado.

2. Identificación y análisis de los interesados

Rol	Nombre y Apellido	Organización	Puesto
Cliente	Juan Schcolnik	-	DJ
Responsable	Ing. Facundo Rivas	FIUBA	Alumno
Orientador	Esp. Ing. Agustín de la Vega	FIUBA	Director del Trabajo Final
Usuario final	-	-	DJs y productores

- Cliente: Juan Schcolnik, es el encargado de aprobar el entregable final.
- Responsable: el Ing. Facundo Rivas, es quien llevará a cabo el desarrollo del proyecto.
- Orientador: el Esp. Ing. Agustín de la Vega, será quien proporcione la ayuda técnica necesaria.
- Usuario final: el producto esta orientado a DJs y productores de música electrónica.

3. Propósito del proyecto

Desarrollar una herramienta innovadora que automatice la clasificación de subgéneros de música electrónica, permitiendo a DJs, productores y profesionales del sector musical organizar eficientemente sus bibliotecas musicales sin dedicar tiempo a la categorización manual. Se busca lograr un sistema con precisión superior al 80 % que distinga entre ocho subgéneros del espectro *techno/house*, ofreciendo una solución completa que combine un modelo de *deep learning* robusto con una aplicación de escritorio funcional y accesible. Con este proyecto se pretende, además, responder a la creciente necesidad de automatización en la gestión de grandes colecciones musicales y facilitar que los artistas se concentren en su labor creativa principal en lugar de tareas administrativas repetitivas.

4. Alcance del proyecto

El proyecto incluye:

- Construcción del dataset: recolección y validación de 1600 *tracks* de música electrónica (200 por cada uno de 8 subgéneros) desde plataformas reconocidas como Beatport y Soundcloud.
- Desarrollo del modelo de clasificación:
 - Implementación de *pipeline* de extracción de características de audio utilizando Librosa (*mel-spectrograms*, *MFCCs*, *tempograms*, *spectral features*).
 - Diseño e implementación de arquitectura *Short-chunk* CNN con ResNet.
 - Entrenamiento y optimización del modelo mediante *hyperparameter tuning*.
 - Validación del modelo con métricas de desempeño (*accuracy*, *F1-score*, matriz de confusión).

- Desarrollo de aplicación de escritorio:
 - Implementación de interfaz gráfica multiplataforma con Electron y React.
 - Integración del modelo de clasificación con el *frontend*.
 - Funcionalidad de selección de directorios y procesamiento *batch*.
 - Sistema de organización automática de archivos por subgénero.
 - Visualización de progreso y resultados con nivel de confianza.
- Pruebas y validación:
 - Pruebas de funcionamiento del sistema completo.
 - Evaluación de métricas de desempeño del modelo.
 - Pruebas de usabilidad de la aplicación.
- Documentación técnica:
 - Manual de usuario de la aplicación.
 - Documentación del código y arquitectura del sistema.
 - Informe de avance del proyecto.
 - Memoria técnica del trabajo final.

El presente proyecto no incluye:

- Clasificación en tiempo real durante reproducción de audio.
- Desarrollo de interfaz web o aplicación móvil.
- Soporte para formatos de video o visualizaciones de audio.
- Clasificación de géneros fuera del espectro *techno/house* seleccionado.
- Análisis de metadatos (artista, sello discográfico, año) para la clasificación.
- Sistema de recomendación de música basado en las clasificaciones.
- Integración con plataformas de *streaming* (Spotify, Beatport, etc.).
- Almacenamiento en la nube o gestión de bibliotecas musicales *online*.
- Soporte técnico o mantenimiento posterior a la entrega del proyecto.
- Entrenamiento del modelo con datos generados sintéticamente.

5. Supuestos del proyecto

Para el desarrollo del presente proyecto se supone:

- Disponibilidad de datos: se tendrá acceso a las plataformas Beatport y Soundcloud para la descarga de los 1600 *tracks* necesarios, con clasificaciones validadas profesionalmente por la comunidad de música electrónica.

- Disponibilidad de recursos computacionales: se contará con acceso a hardware adecuado (GPU) para el entrenamiento del modelo de *deep learning*, ya sea de forma local o mediante servicios en la nube.
- Factibilidad técnica: las tecnologías actuales de procesamiento de audio y *deep learning* son suficientemente avanzadas para alcanzar el objetivo de 80 % de *accuracy* en la clasificación de los ocho subgéneros seleccionados.
- Tiempo de desarrollo: se estima que el tiempo asignado para el proyecto será suficiente para completar todas las etapas, incluido la construcción del dataset, desarrollo del modelo, implementación de la aplicación y pruebas del sistema.
- Calidad del dataset: los *tracks* obtenidos de las plataformas mencionadas tendrán calidad de audio suficiente (*bitrate* mínimo) para la extracción efectiva de características espectrales y temporales.
- Consistencia en clasificaciones: existe consenso suficiente en la industria sobre las características que definen cada uno de los ocho subgéneros seleccionados, lo que permite la validación cruzada entre múltiples fuentes.
- Consideraciones legales: el uso de archivos de audio para entrenamiento del modelo bajo el marco de investigación académica es aceptable, sin distribución de los archivos originales.

6. Requerimientos

1. Requerimientos funcionales del modelo de clasificación:

- 1.1. El sistema debe ser capaz de procesar archivos de audio en formatos MP3, AIFF y WAV.
- 1.2. El modelo debe clasificar *tracks* en exactamente 8 subgéneros: *deep house*, *tech house*, *melodic techno*, *progressive*, *techno peak time*, *hard techno*, *minimal/deep tech* y *trance*.
- 1.3. El sistema debe alcanzar un *accuracy* general superior al 80 % en el conjunto de *test*.
- 1.4. El sistema debe alcanzar un top-2 *accuracy* superior al 90 % (considerando las dos clases más probables).
- 1.5. El modelo debe procesar cada *track* en menos de 5 segundos en hardware con GPU dedicada.
- 1.6. El sistema debe proporcionar el nivel de confianza (probabilidad) para cada predicción realizada.
- 1.7. El modelo debe ser capaz de extraer y utilizar las siguientes características de audio: *mel-spectrograms*, *MFCCs*, *tempograms*, *spectral centroid*, *spectral rolloff*, *zero crossing rate* y *chroma features*.

2. Requerimientos funcionales de la aplicación de escritorio:

- 2.1. La aplicación debe permitir al usuario seleccionar un directorio que contenga archivos de audio para clasificar.
- 2.2. El sistema debe mostrar el progreso de procesamiento en tiempo real indicando cantidad de archivos procesados y tiempo estimado restante.
- 2.3. La aplicación debe crear automáticamente subdirectorios con el nombre de cada subgénero identificado.

- 2.4. El sistema debe mover cada archivo clasificado al subdirectorío correspondiente a su subgénero predicho.
- 2.5. La aplicación debe mostrar un reporte final con estadísticas: cantidad de archivos por subgénero, nivel de confianza promedio y tiempo total de procesamiento.
- 2.6. El sistema debe permitir procesamiento por lotes (*batch*) de múltiples archivos sin intervención del usuario.
- 2.7. La aplicación debe ser compatible con sistemas operativos Windows, macOS y Linux.
3. Requerimientos del dataset:
 - 3.1. El dataset debe contener un mínimo de 1600 *tracks* distribuidos equitativamente: 200 por cada uno de los 8 subgéneros.
 - 3.2. Cada *track* del dataset debe tener su clasificación validada por al menos dos fuentes profesionales (Beatport, Soundcloud u otras plataformas reconocidas).
 - 3.3. Los archivos de audio del dataset deben tener una calidad mínima de 128 kbps.
 - 3.4. El dataset debe dividirse en 70 % *train*, 15 % *test* y 15 % *validation*, manteniendo balance entre clases.
4. Requerimientos de documentación:
 - 4.1. Se debe entregar un manual de usuario que explique la instalación y uso de la aplicación de escritorio.
 - 4.2. Se debe documentar la arquitectura del sistema incluyendo diagramas de componentes y flujo de datos.
 - 4.3. Se debe documentar el código fuente siguiendo estándares de Python (PEP 8) y JavaScript.
 - 4.4. Se debe elaborar un informe de avance del proyecto.
 - 4.5. Se debe elaborar una memoria técnica del trabajo final que incluya resultados experimentales y análisis de desempeño.
5. Requerimientos de diseño:
 - 5.1. El módulo de clasificación (Python) y la interfaz gráfica (Electron/React) deben ser componentes independientes comunicados mediante API.
 - 5.2. La arquitectura debe permitir el reemplazo del modelo de clasificación sin modificar la interfaz gráfica.
 - 5.3. El código debe estar modularizado para facilitar futuras extensiones a nuevos subgéneros.
6. Requerimientos de *testing* y *validation*:
 - 6.1. Se deben realizar pruebas unitarias del módulo de extracción de características con cobertura mínima del 70 %.
 - 6.2. Se debe generar y analizar la matriz de confusión del modelo sobre el conjunto de *test*.
 - 6.3. Se debe calcular el *F1-score* para cada clase y reportar el *F1-score* macro promedio.
 - 6.4. Se deben realizar pruebas de usabilidad de la aplicación con al menos un usuario final (DJ o productor).
7. Requerimientos no funcionales:

- 7.1. El sistema no debe almacenar en disco permanente ningún archivo de audio procesado por el usuario (solo el resultado final organizado).
 - 7.2. El código fuente debe estar versionado utilizando Git.
 - 7.3. El tiempo de inicio de la aplicación no debe superar los 10 segundos.
8. Requerimientos legales y éticos:
- 8.1. El uso de archivos de audio del dataset se limitará exclusivamente a fines de investigación académica, sin distribución pública de los archivos originales.
 - 8.2. Se respetarán los derechos de autor, utilizando únicamente el análisis de características extraídas y no los archivos de audio completos en publicaciones.

Nota sobre prioridades: los requerimientos funcionales del modelo de clasificación (grupo 1) y de la aplicación (grupo 2) son de máxima prioridad y deben cumplirse para considerar el proyecto exitoso. Los requerimientos de *testing* (grupo 6) son de alta prioridad. Los requerimientos de documentación (grupo 4) tienen prioridad media pero son obligatorios para la entrega final.

7. Historias de usuarios (*Product backlog*)

7.1. Criterio de ponderación

Se definen los siguientes roles:

- Usuario final: DJs y productores de música electrónica que necesitan organizar sus bibliotecas musicales. No requieren conocimientos técnicos de inteligencia artificial, solo deben proporcionar sus archivos de audio.
- Desarrollador: persona con conocimientos en *deep learning*, procesamiento de audio y desarrollo de aplicaciones, responsable de implementar el sistema completo.
- Cliente: persona encargada de verificar que el proyecto cumple con las especificaciones y aprobar el entregable final.

Se llevarán a cabo *sprints* de 2 semanas, donde cada *sprint* tendrá una dedicación de 40 horas. La complejidad, dificultad e incertidumbre de cada historia de usuario se ponderará utilizando los primeros 5 términos (sin contar el 0) de la serie de Fibonacci: 1, 2, 3, 5, 8. Las equivalencias de estos puntos son las siguientes:

- 1 punto: historia de usuario estimada a ser resuelta en pocas horas. Generalmente se trata de implementaciones simples o correcciones menores.
- 2 puntos: historia de usuario estimada a ser resuelta en un día laboral.
- 3 puntos: historia de usuario estimada a ser resuelta en hasta tres días laborales.
- 5 puntos: historia de usuario estimada a ser resuelta en una semana laboral.
- 8 puntos: historia de usuario estimada a ser resuelta en un *sprint* completo.

Sumando estos 3 términos (complejidad + dificultad + incertidumbre) se llegará a la estimación final de *story points*.

7.2. Historias de usuario

1. Como desarrollador quiero construir un dataset balanceado de 1600 *tracks* con clasificaciones validadas para entrenar el modelo de clasificación de manera robusta.
Story points: 10 (complejidad: 3, dificultad: 5, incertidumbre: 2)
2. Como desarrollador quiero implementar un *pipeline* de extracción de características de audio (*mel-spectrograms*, *MFCCs*, *tempograms*, etc.) para obtener representaciones numéricas de los *tracks*.
Story points: 13 (complejidad: 5, dificultad: 5, incertidumbre: 3)
3. Como desarrollador quiero diseñar y entrenar una arquitectura *Short-chunk* CNN con ResNet para clasificar los 8 subgéneros con alta precisión.
Story points: 21 (complejidad: 8, dificultad: 8, incertidumbre: 5)
4. Como desarrollador quiero optimizar los hiperparámetros del modelo mediante *hyperparameter tuning* para maximizar el *accuracy* y minimizar el *overfitting*.
Story points: 13 (complejidad: 5, dificultad: 5, incertidumbre: 3)
5. Como desarrollador quiero evaluar el modelo con métricas cuantitativas (*accuracy*, *F1-score*, matriz de confusión) para validar su desempeño.
Story points: 8 (complejidad: 3, dificultad: 3, incertidumbre: 2)
6. Como desarrollador quiero implementar una aplicación de escritorio con Electron y React para proporcionar una interfaz amigable al usuario.
Story points: 16 (complejidad: 5, dificultad: 5, incertidumbre: 6)
7. Como desarrollador quiero integrar el modelo de clasificación con la aplicación de escritorio mediante una API para que funcionen de manera coordinada.
Story points: 11 (complejidad: 5, dificultad: 3, incertidumbre: 3)
8. Como usuario final quiero seleccionar un directorio con mis *tracks* y que el sistema los clasifique automáticamente para ahorrar tiempo en la organización manual.
Story points: 8 (complejidad: 3, dificultad: 3, incertidumbre: 2)
9. Como usuario final quiero ver el progreso de clasificación en tiempo real para saber cuánto tiempo falta para completar el proceso.
Story points: 5 (complejidad: 2, dificultad: 2, incertidumbre: 1)
10. Como usuario final quiero que el sistema cree automáticamente carpetas por subgénero y mueva los archivos clasificados para tener mi biblioteca organizada sin intervención manual.
Story points: 6 (complejidad: 2, dificultad: 2, incertidumbre: 2)
11. Como usuario final quiero ver un reporte con estadísticas de la clasificación (cantidad por género, confianza promedio) para evaluar los resultados obtenidos.
Story points: 5 (complejidad: 2, dificultad: 2, incertidumbre: 1)
12. Como cliente quiero que el sistema alcance un *accuracy* superior al 80 % y un top-2 *accuracy* superior al 90 % para garantizar clasificaciones confiables.
Story points: 21 (complejidad: 8, dificultad: 8, incertidumbre: 5)

13. Como cliente quiero recibir documentación técnica completa (manual de usuario, arquitectura del sistema, código documentado) para comprender el funcionamiento y facilitar el mantenimiento futuro.

Story points: 8 (complejidad: 5, dificultad: 2, incertidumbre: 1)

8. Entregables principales del proyecto

Los entregables del proyecto son:

- Código fuente del módulo de clasificación (Python), incluido:
 - *Scripts* de construcción y preprocesamiento del dataset.
 - *Pipeline* de extracción de características de audio.
 - Implementación de la arquitectura *Short-chunk* CNN con ResNet.
 - *Scripts* de entrenamiento y evaluación del modelo.
- Modelo entrenado de clasificación en formato serializado, listo para inferencia.
- Código fuente de la aplicación de escritorio (Electron + React), incluido:
 - Interfaz gráfica de usuario.
 - Integración con el modelo de clasificación.
 - Sistema de organización automática de archivos.
- Aplicación de escritorio empaquetada y lista para instalación en sistemas Windows, macOS y Linux.
- Dataset de validación con 240 *tracks* (15 % del total) debidamente etiquetados.
- Resultados de evaluación del modelo, incluido:
 - Métricas de desempeño (*accuracy*, *top-2 accuracy*, *F1-score*).
 - Matriz de confusión detallada.
 - Análisis de errores y confusiones sistemáticas entre subgéneros.
- Manual de usuario que explique:
 - Requisitos del sistema e instalación de la aplicación.
 - Guía paso a paso para clasificar *tracks*.
 - Interpretación de resultados y nivel de confianza.
 - Resolución de problemas comunes.
- Documentación técnica del sistema, que incluya:
 - Diagrama de arquitectura del sistema completo.
 - Especificación de la API de comunicación entre módulos.
 - Documentación del código fuente siguiendo estándares (PEP 8, JSDoc).
 - Descripción de la arquitectura del modelo de *deep learning*.
- Informe de avance del proyecto con el estado de desarrollo a mitad del plazo establecido.

- Memoria técnica del trabajo final que incluya:
 - Metodología de construcción del dataset.
 - Proceso de diseño, entrenamiento y optimización del modelo.
 - Resultados experimentales y análisis comparativo.
 - Conclusiones y trabajo futuro.

9. Desglose del trabajo en tareas

1. Planificación del proyecto (70 h)
 - 1.1. Elaboración del plan de trabajo (40 h)
 - 1.2. Selección de herramientas y tecnologías (15 h)
 - 1.3. Diseño de la arquitectura del sistema (15 h)
2. Construcción del dataset (80 h)
 - 2.1. Investigación y selección de fuentes de datos (10 h)
 - 2.2. Descarga y recolección de *tracks* de plataformas (40 h)
 - 2.3. Validación cruzada de clasificaciones (20 h)
 - 2.4. Organización y etiquetado del dataset (10 h)
3. Módulo de extracción de características (60 h)
 - 3.1. Capacitación en procesamiento de audio con Librosa (15 h)
 - 3.2. Implementación de extracción de *mel-spectrograms* y *MFCCs* (15 h)
 - 3.3. Implementación de extracción de *tempograms* y *spectral features* (15 h)
 - 3.4. Pruebas y validación del *pipeline* de extracción (10 h)
 - 3.5. Documentación del módulo (5 h)
4. Desarrollo del modelo de clasificación (120 h)
 - 4.1. Capacitación en arquitecturas CNN con ResNet (20 h)
 - 4.2. Implementación de la arquitectura *Short-chunk* CNN (30 h)
 - 4.3. Preparación de datos para entrenamiento (división *train/validation/test*) (10 h)
 - 4.4. Entrenamiento del modelo base (30 h)
 - 4.5. *Hyperparameter tuning* y optimización (20 h)
 - 4.6. Pruebas y evaluación del modelo (5 h)
 - 4.7. Documentación del modelo (5 h)
5. Desarrollo de la aplicación de escritorio (100 h)
 - 5.1. Capacitación en Electron y React (15 h)
 - 5.2. Diseño de la interfaz gráfica de usuario (15 h)
 - 5.3. Implementación del selector de directorios (10 h)
 - 5.4. Implementación de la visualización de progreso en tiempo real (15 h)
 - 5.5. Implementación del sistema de organización automática de archivos (20 h)

- 5.6. Implementación del reporte de resultados con estadísticas (10 h)
- 5.7. Pruebas de usabilidad (10 h)
- 5.8. Documentación de la aplicación (5 h)
- 6. Integración y empaquetado (50 h)
 - 6.1. Diseño e implementación de API entre módulos (20 h)
 - 6.2. Integración del modelo de clasificación con la aplicación (15 h)
 - 6.3. Empaquetado para Windows, macOS y Linux (10 h)
 - 6.4. Pruebas de integración del sistema completo (5 h)
- 7. Pruebas y validación (50 h)
 - 7.1. Pruebas unitarias del módulo de extracción (10 h)
 - 7.2. Evaluación completa del modelo (métricas, matriz de confusión) (15 h)
 - 7.3. Pruebas con usuario final (DJ/productor) (15 h)
 - 7.4. Ajustes y correcciones basados en retroalimentación (10 h)
- 8. Cierre del proyecto (70 h)
 - 8.1. Elaboración del informe de avance (10 h)
 - 8.2. Elaboración del manual de usuario (15 h)
 - 8.3. Elaboración de la documentación técnica (15 h)
 - 8.4. Elaboración de la memoria técnica del trabajo final (25 h)
 - 8.5. Preparación de la presentación final (5 h)

Cantidad total de horas: 600 h.

11. Diagrama de Gantt

En las figuras 3 y 4 se presenta el diagrama de Gantt del proyecto, donde se visualiza la planificación temporal de todas las tareas identificadas en el WBS. Las barras representan la duración de cada tarea y las agrupaciones muestran los 8 grupos principales del proyecto. El diagrama permite identificar las actividades que se ejecutan en paralelo y aquellas que siguen una secuencia obligatoria. La fase de planificación se realiza durante la cursada de Gestión de Proyectos (octubre-diciembre 2025), el desarrollo del sistema se ejecuta entre diciembre 2025 y marzo 2026, y la elaboración de la memoria técnica se realiza durante la cursada del Taller de Trabajo Final (mayo-julio 2026).

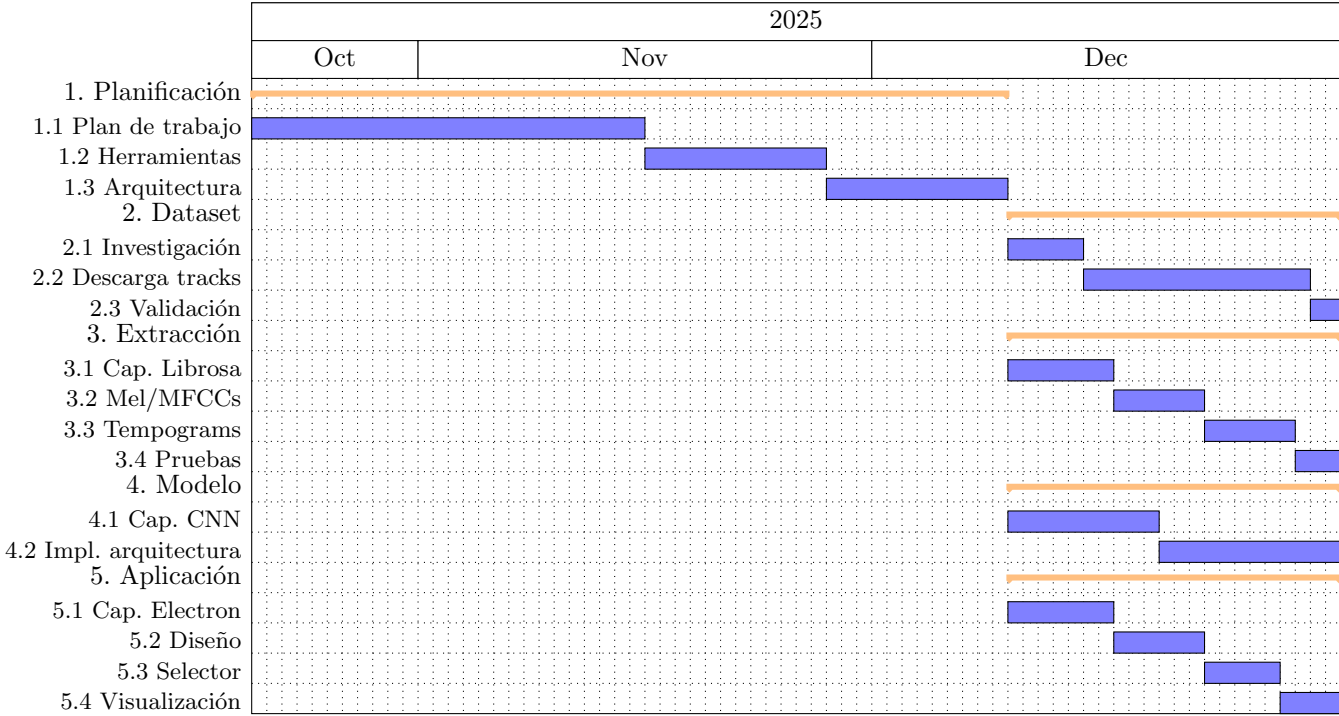


Figura 3. Diagrama de Gantt del proyecto - Año 2025.

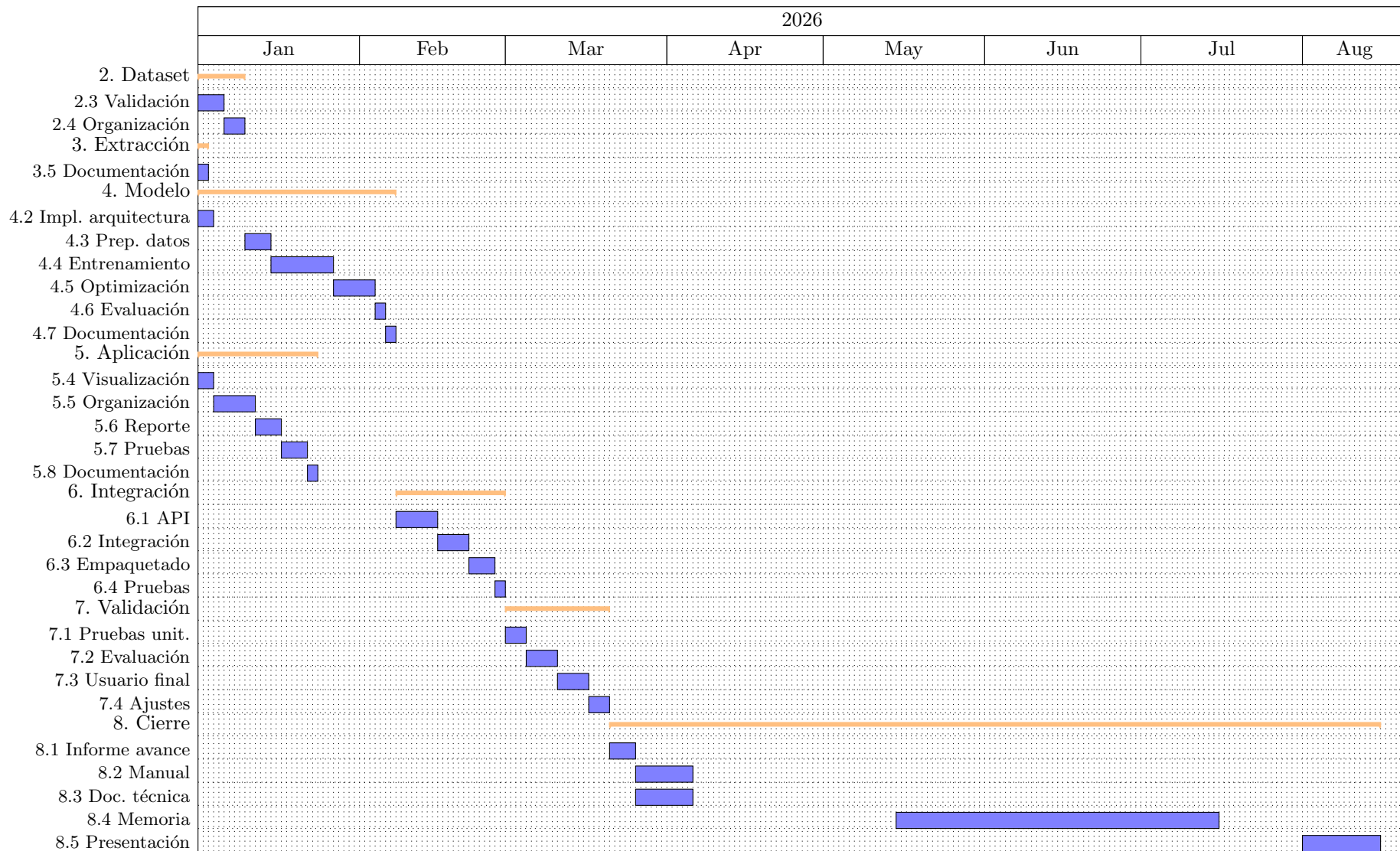


Figura 4. Diagrama de Gantt del proyecto - Año 2026.

12. Presupuesto detallado del proyecto

El presupuesto del proyecto se expresa en dólares estadounidenses (USD). La tasa de conversión al peso argentino al 18 de noviembre de 2025 es de 1 USD = 1435 ARS (valor referencial).

COSTOS DIRECTOS			
Descripción	Cantidad	Valor unitario	Valor total
Horas de ingeniería (desarrollo y entrenamiento)	600 h	20,00 USD	12.000,00 USD
Instancia de procesamiento en la nube (GPU)	200 h	0,75 USD	150,00 USD
Computadora de desarrollo	1 u	1.200,00 USD	1.200,00 USD
Licencias de software y servicios (Beatport, APIs)	1 u	150,00 USD	150,00 USD
Almacenamiento en la nube (dataset y modelos)	100 GB	0,50 USD	50,00 USD
SUBTOTAL			13.550,00 USD
COSTOS INDIRECTOS			
Descripción	Cantidad	Valor unitario	Valor total
Overhead administrativo (20 % del costo directo)	-	-	2.710,00 USD
Contingencias e imprevistos (10 % del costo directo)	-	-	1.355,00 USD
Energía eléctrica e internet	4 meses	50,00 USD	200,00 USD
SUBTOTAL			4.265,00 USD
TOTAL			17.815,00 USD

Cuadro 1. Presupuesto detallado del proyecto.

13. Gestión de riesgos

a) Identificación de los riesgos y estimación de sus consecuencias:

Riesgo 1: Datos insuficientes o de baja calidad para el entrenamiento del modelo.

- Severidad (S): 8.
La falta de datos variados y correctamente clasificados impactaría directamente en la capacidad del modelo para alcanzar el objetivo de 80% de *accuracy*. Un dataset desbalanceado o con clasificaciones erróneas produciría un modelo poco confiable.
- Probabilidad de ocurrencia (O): 5.
Aunque las plataformas Beatport y Soundcloud tienen catálogos extensos, existe la posibilidad de que algunas clasificaciones no sean consistentes entre fuentes o que ciertos subgéneros tengan menor disponibilidad de *tracks*.

Riesgo 2: Falta de disponibilidad de recursos de procesamiento GPU.

- Severidad (S): 7.
La falta de acceso a recursos de GPU en la nube retrasaría significativamente las fases

de entrenamiento y optimización del modelo. En caso extremo, se podría entrenar con recursos limitados, pero extendiendo los tiempos considerablemente.

- Probabilidad de ocurrencia (O): 3.
Los proveedores de servicios en la nube (AWS, GCP, Azure) son confiables y tienen alta disponibilidad. La principal causa de indisponibilidad sería falta de presupuesto o problemas de pago.

Riesgo 3: El modelo no alcanza el objetivo de 80 % de *accuracy*.

- Severidad (S): 9.
No alcanzar la precisión objetivo comprometería la utilidad práctica del sistema. Un modelo con baja precisión generaría clasificaciones poco confiables, reduciendo el valor del producto final.
- Probabilidad de ocurrencia (O): 4.
La literatura reciente demuestra que es factible alcanzar estos niveles de precisión con arquitecturas CNN. Sin embargo, la distinción entre subgéneros similares (ej: *Melodic Techno* vs *Progressive*) presenta desafíos inherentes.

Riesgo 4: Problemas de integración entre el modelo Python y la aplicación Electron/React.

- Severidad (S): 6.
Dificultades en la comunicación entre módulos retrasarían la fase de integración y podrían requerir rediseño de la arquitectura de comunicación.
- Probabilidad de ocurrencia (O): 5.
La integración entre tecnologías diferentes (Python y JavaScript/Electron) presenta desafíos técnicos conocidos, especialmente en el manejo de procesos y transferencia de datos.

Riesgo 5: Falta de disponibilidad del cliente o usuario final para validación.

- Severidad (S): 5.
Sin validación con usuarios reales (DJs/productores), no se podría verificar la usabilidad del sistema ni obtener retroalimentación valiosa para ajustes finales.
- Probabilidad de ocurrencia (O): 4.
Los profesionales de la música tienen agendas ajustadas, lo que dificulta coordinar sesiones de prueba.

Riesgo 6: Cambios en las APIs o políticas de las plataformas de música.

- Severidad (S): 6.
Modificaciones en los términos de uso o APIs de Beatport/Soundcloud podrían dificultar la obtención de datos o invalidar parte del dataset construido.
- Probabilidad de ocurrencia (O): 3.
Las plataformas establecidas no suelen hacer cambios drásticos frecuentemente, pero es un riesgo a considerar dado el tiempo del proyecto.

b) Tabla de gestión de riesgos: (El RPN se calcula como $RPN = S \times O$)

Riesgo	S	O	RPN	S*	O*	RPN*
1. Datos insuficientes o de baja calidad	8	5	40	8	2	16
2. Falta de disponibilidad de GPU	7	3	21	-	-	-
3. Modelo no alcanza 80 % accuracy	9	4	36	6	3	18
4. Problemas de integración Python/Electron	6	5	30	6	2	12
5. Falta de disponibilidad para validación	5	4	20	-	-	-
6. Cambios en APIs de plataformas	6	3	18	-	-	-

Cuadro 2. Tabla de gestión de riesgos.

Criterio adoptado: Se tomarán medidas de mitigación en los riesgos cuyos números de RPN sean mayores a 25.

Nota: los valores marcados con (*) en la tabla corresponden luego de haber aplicado la mitigación.

c) Plan de mitigación de los riesgos que originalmente excedían el RPN máximo establecido:

Riesgo 1: Datos insuficientes o de baja calidad para el entrenamiento del modelo.

Plan de mitigación: Se implementará un proceso riguroso de validación cruzada donde cada *track* debe tener clasificación consistente en al menos dos fuentes diferentes. Además, se establecerá un criterio de calidad mínima de audio (128 kbps) y se descartarán *tracks* con clasificaciones ambiguas. Se mantendrá un margen de 20 % adicional en la recolección inicial (1920 *tracks*) para compensar posibles descartes.

- Severidad (S*): 8.
La severidad se mantiene igual ya que el impacto de datos de mala calidad sigue siendo alto.
- Probabilidad de ocurrencia (O*): 2.
Con el proceso de validación cruzada y el margen adicional, la probabilidad de tener un dataset insuficiente se reduce significativamente.

Riesgo 3: El modelo no alcanza el objetivo de 80 % de *accuracy*.

Plan de mitigación: Se implementará un enfoque iterativo de desarrollo del modelo, comenzando con una arquitectura base y realizando mejoras incrementales. Se evaluarán múltiples arquitecturas (no solo Short-chunk CNN) y se aplicará *transfer learning* desde modelos preentrenados en tareas de audio. Como contingencia, se definirá un objetivo secundario de top-2 *accuracy* del 90 % que sería aceptable si el *accuracy* simple queda entre 75-80 %.

- Severidad (S*): 6.
Con el objetivo secundario definido, el impacto de no alcanzar exactamente el 80 % se reduce, ya que el sistema seguiría siendo útil.
- Probabilidad de ocurrencia (O*): 3.
La evaluación de múltiples arquitecturas y el uso de *transfer learning* aumentan las probabilidades de alcanzar el objetivo.

Riesgo 4: Problemas de integración entre el modelo Python y la aplicación Electron/React.

Plan de mitigación: Se diseñará la API de comunicación entre módulos desde las etapas

tempranas del proyecto, definiendo claramente los contratos de interfaz. Se realizarán pruebas de integración incrementales durante el desarrollo (no solo al final). Como alternativa técnica, se evaluará el uso de un servidor local Flask/FastAPI para la comunicación, en lugar de llamadas directas a Python desde Electron.

- Severidad (S^*): 6.
La severidad se mantiene ya que los problemas de integración seguirían causando retrasos.
- Probabilidad de ocurrencia (O^*): 2.
El diseño temprano de la API y las pruebas incrementales reducen significativamente la probabilidad de problemas graves de integración.

14. Gestión de la calidad

- Req #1.1: El sistema debe ser capaz de procesar archivos de audio en formatos MP3, AIFF y WAV.
 - Verificación: se realizarán pruebas unitarias del módulo de carga de archivos utilizando un conjunto de prueba con al menos 10 archivos de cada formato, verificando que el sistema los procesa sin errores y extrae correctamente la señal de audio.
 - Validación: el cliente proporcionará archivos de su biblioteca personal en los tres formatos y confirmará que el sistema los procesa correctamente.
- Req #1.3: El sistema debe alcanzar un *accuracy* general superior al 80 % en el conjunto de test.
 - Verificación: se evaluará el modelo entrenado sobre el conjunto de test (15 % del dataset, 240 *tracks*) calculando la métrica de *accuracy* y documentando los resultados en un informe técnico.
 - Validación: el cliente revisará el informe de evaluación con la matriz de confusión y confirmará que el *accuracy* reportado cumple con el objetivo establecido.
- Req #1.4: El sistema debe alcanzar un top-2 *accuracy* superior al 90 %.
 - Verificación: se calculará el top-2 *accuracy* sobre el conjunto de test, verificando que para al menos el 90 % de los *tracks* el subgénero correcto está entre las dos predicciones más probables.
 - Validación: el cliente seleccionará 20 *tracks* de su biblioteca con clasificación conocida y verificará que el sistema presenta el género correcto entre las dos opciones principales.
- Req #1.5: El modelo debe procesar cada *track* en menos de 5 segundos en hardware con GPU dedicada.
 - Verificación: se medirá el tiempo de procesamiento de 100 *tracks* de diferentes duraciones, calculando el tiempo promedio y máximo, documentando las especificaciones del hardware utilizado.
 - Validación: el cliente ejecutará la clasificación de una carpeta con al menos 50 *tracks* y verificará que el tiempo total es consistente con el límite de 5 segundos por archivo.

- Req #2.1: La aplicación debe permitir al usuario seleccionar un directorio que contenga archivos de audio para clasificar.
 - Verificación: se probará el componente de selección de directorios en los tres sistemas operativos (Windows, macOS, Linux), verificando que navega correctamente el sistema de archivos y permite seleccionar carpetas.
 - Validación: el cliente utilizará la aplicación para seleccionar diferentes directorios de su computadora y confirmará que la interfaz es intuitiva y funcional.
- Req #2.3: La aplicación debe crear automáticamente subdirectorios con el nombre de cada subgénero identificado.
 - Verificación: se ejecutará el sistema con un conjunto de prueba y se verificará programáticamente que se crean exactamente 8 subdirectorios con los nombres correctos de los subgéneros.
 - Validación: el cliente ejecutará la clasificación sobre una carpeta de prueba y confirmará que los subdirectorios creados tienen nombres claros y corresponden a los subgéneros esperados.
- Req #2.4: El sistema debe mover cada archivo clasificado al subdirectorio correspondiente a su subgénero predicho.
 - Verificación: se ejecutará el sistema con 50 archivos de prueba y se verificará que cada archivo se encuentra en el subdirectorio correspondiente a su predicción, sin pérdida ni corrupción de datos.
 - Validación: el cliente clasificará una carpeta con *tracks* conocidos y verificará manualmente que los archivos fueron movidos a las carpetas correctas según la predicción del sistema.
- Req #2.7: La aplicación debe ser compatible con sistemas operativos Windows, macOS y Linux.
 - Verificación: se instalará y ejecutará la aplicación empaquetada en al menos una versión de cada sistema operativo (Windows 10/11, macOS 12+, Ubuntu 22.04), documentando el proceso y verificando todas las funcionalidades.
 - Validación: el cliente instalará la aplicación en su sistema operativo habitual y confirmará que funciona correctamente sin errores de compatibilidad.
- Req #3.1: El dataset debe contener un mínimo de 1600 *tracks* distribuidos equitativamente (200 por subgénero).
 - Verificación: se contabilizará el dataset final mediante un *script* que verifique la cantidad de archivos por carpeta/etiqueta, generando un reporte con la distribución exacta.
 - Validación: el cliente revisará el reporte de distribución del dataset y confirmará que cumple con los requisitos de cantidad y balance entre clases.
- Req #5.1: El módulo de clasificación (Python) y la interfaz gráfica (Electron/React) deben ser componentes independientes comunicados mediante API.
 - Verificación: se ejecutará el módulo de clasificación de forma independiente mediante línea de comandos, verificando que produce resultados correctos sin necesidad de la interfaz gráfica. Se documentará la especificación de la API.

- Validación: el cliente recibirá documentación técnica de la API y podrá verificar que ambos componentes pueden ejecutarse y probarse de forma independiente.
- Req #6.2: Se debe generar y analizar la matriz de confusión del modelo sobre el conjunto de test.
 - Verificación: se generará la matriz de confusión utilizando el conjunto de test completo, calculando métricas derivadas (*precision*, *recall*, *F1-score* por clase) y analizando los pares de subgéneros con mayor confusión.
 - Validación: el cliente revisará la matriz de confusión y el análisis de errores, confirmando que los resultados son comprensibles y que las confusiones identificadas son razonables desde el punto de vista musical.
- Req #4.1: Se debe entregar un manual de usuario que explique la instalación y uso de la aplicación de escritorio.
 - Verificación: se revisará que el manual incluya todas las secciones requeridas: requisitos del sistema, instrucciones de instalación, guía de uso paso a paso, interpretación de resultados y resolución de problemas comunes.
 - Validación: el cliente seguirá las instrucciones del manual para instalar y utilizar la aplicación sin asistencia adicional, reportando cualquier paso confuso o faltante.

15. Procesos de cierre

Para dar como completado el proyecto, se llevarán a cabo las siguientes actividades:

- Pautas de trabajo que se seguirán para analizar si se respetó el Plan de Proyecto original:
 - Responsable: Ing. Facundo Rivas.
 - Procedimiento: se realizará una comparación exhaustiva entre el plan de proyecto original y la ejecución real. Se analizará el grado de cumplimiento de todas las tareas programadas, comparando el tiempo dedicado a cada una de ellas contra las estimaciones originales. Se evaluará si se alcanzaron los objetivos de *accuracy* (80 % general, 90 % top-2) y si los entregables cumplen con los requerimientos establecidos. En caso de encontrar tareas sin finalizar o desvíos significativos con sus estimaciones, se documentarán las causas y se elaborará un análisis de lecciones aprendidas. Los resultados se plasmarán en un informe de cierre que será incluido como anexo en la memoria técnica del trabajo final.
- Identificación de las técnicas y procedimientos útiles e inútiles que se emplearon, los problemas que surgieron y cómo se solucionaron:
 - Responsable: Ing. Facundo Rivas.
 - Procedimiento: se elaborará un documento de lecciones aprendidas que incluirá las siguientes secciones:
 - Técnicas de desarrollo que resultaron efectivas (arquitecturas de modelo, herramientas de extracción de características, frameworks utilizados).
 - Técnicas que no aportaron valor o fueron descartadas durante el desarrollo.
 - Problemas técnicos encontrados (entrenamiento del modelo, integración de módulos, compatibilidad multiplataforma) y las soluciones implementadas.

- Desafíos en la construcción del dataset y cómo se resolvieron.
- Recomendaciones para proyectos futuros similares.

Este documento quedará registrado en el repositorio del proyecto y será referenciado en la memoria técnica para beneficio de futuros desarrolladores o investigadores que trabajen en clasificación de música electrónica.

- Organización del acto de agradecimiento a todos los interesados, y en especial al equipo de trabajo y colaboradores:
 - Responsable: Ing. Facundo Rivas.
 - Procedimiento: se organizará una reunión virtual de cierre con todos los interesados del proyecto (director del trabajo final, cliente, colaboradores que participaron en las pruebas de validación). En esta reunión se presentarán los resultados finales del proyecto, se realizará una demostración del sistema funcionando y se reconocerá el aporte de cada una de las partes en las diferentes etapas. Se enviará un correo de agradecimiento formal a cada participante, destacando su contribución específica al proyecto.
 - Financiamiento: los gastos asociados a la reunión virtual (plataforma de videoconferencia) serán cubiertos por el responsable del proyecto. En caso de realizarse una reunión presencial opcional posterior a la defensa pública, los gastos serán asumidos por el Ing. Facundo Rivas.

Referencias

- [1] Hsu, J. L., Liu, C. C., & Chen, A. L. P. (2021). *Deep Learning Based EDM Subgenre Classification*. Proceedings of the International Conference on Machine Learning and Applications.